C++ Memory Mechanics: Object Slicing, Pointers, & References

Understanding Object Slicing

Eg

```
class Animal {
public:
    int a;
    Animal(int a) {
        this->a = a;
    }
    void show() { }
};

class Dog : public Animal {
public:
    int b;
    Dog(int a, int b) : Animal(a) {
        this->a = a;
        this->b = b;
    }
    void showdog() { }
};
```

now if we do

```
Dog d(1, 2);
Animal a = d;
```

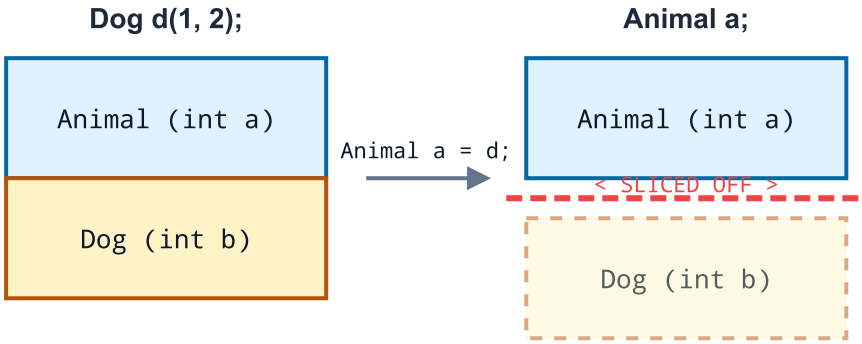
then passing a derived class object into a base class object by value, leads to object slicing. the derived specific portion gets sliced off.

so if we do

- `a.show()` -> ✔
- `a.showdog()` -> ❌

Reasoning & Memory Mechanics

reason: a base class variable allocates enough memory to hold base-class part of an object.



so only `int a` part was read and `b` was ignored. when called `a.showdog()` gave compile error because `Animal` class dont have that method.

Pointer vs Reference (Comparison)

Feature	`Animal *ptr = &d`	`Animal &ref = d`
syntax for access	`ptr->show()`	`ptr.show()`
initialization	can be declared without an object, i.e `Animal *ptr;`	must be initialized immediately upon creation
reassignment	can be changed to point to diff object	permanently bound to `d` and cant change
nullability	can be `nullptr`	cant be null must always refer to valid object
memory	occupies own memory to store address	shares same memory address as the object

Why these differences exist:
these differences exist because pointer is an independent variable that holds memory address of the object it's pointing to. but reference is simply an alias for an existing object it compiled level. It's just a label attached with an object, and we cant attach label to nothing, thats why it need to get initialized.

so `Animal &ref = d;` means [`ref` and `d` are same entity]. whenever we use `ref` it just replaces it with `d` 's address.

now understand

Error Type Mismatch:
`Animal * ptr = d;` -> [Error] [type mismatch]
(`d` is object, `ptr` is pointer)

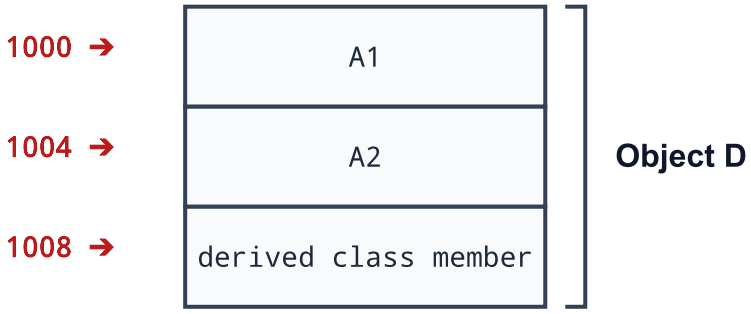
```
Animal * ptr = new Dog();
```

- a dog object is created dynamically
- It returns a `Dog*` which is implicitly converted to `Animal*`
- [`ptr` points to Animal subobject of dynamically allocated dog]

Memory Layout in Multiple Inheritance

now imagine we have multiple inheritance `Animal1` , `Animal2` and `Dog` inherits from both.

Diagram (Object D memory layout):



Pointer Offsets vs By-Value Copying

now if we write `Animal2 * ptr = &d`

then `&d` points to start of object which is `1000`. `Animal2` knows it's at offset of `4` so it adjusts itself to `1004`.

now `ptr` points to `Animal2` portion.

> **Crucial Difference: Pointer Adjustment vs Value Copying**
>
> BUT SINCE NOTHING IS MODIFIED complete dog D is still in the memory we just changed which portion of it the pointer refers to. (i.e. `Animal2 * ptr = &d`).
>
> Earlier [`Animal2 obj = d`]
> we were copying the A2 data from object `d` to `Animal2` so `Animal2` has no info of `Animal1` or dog `d` specific.